Standard Template Library (STL): Introduction, Array, Vector, List, Forward _List, Queue, Priority Queue, Stack

Wattana Jindaluang

The C++ Standard Template Library: STL

- The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, queue, etc.
- It is a library of container classes, algorithms, and iterators
- The C++ Standard Template Library (STL) is a collection of algorithms, data structures, and other components that can be used to simplify the development of C++ programs.

The C++ Standard Template Library: STL (Cont.)

- The STL provides a range of containers, such as vectors, lists, and maps, as well as algorithms for searching, sorting and manipulating data.
- The STL also provides a way to write efficient code. Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.

The C++ Standard Template Library: STL (Cont.)

- Some of the key components of the STL include:
  - Containers: The STL provides a range of containers, such as vector, list, map, set, and stack, which can be used to store and manipulate data.
  - Algorithms: The STL provides a range of algorithms, such as sort, find, and binary_search, which can be used to manipulate data stored in containers.
  - Iterators: Iterators are objects that provide a way to traverse the elements of a container. The STL provides a range of iterators, such as forward_iterator, bidirectional_iterator, and random_access_iterator, that can be used with different types of containers.

The C++ Standard Template Library: STL (Cont.)

- Containers
  - Sequence Containers: implement data structures that can be accessed in a sequential manner. -> vector, list, arrays and forward_list
  - Container Adaptors: provide a different interface for sequential containers -> queue, priority_queue, stack
  - Associative Containers: implement sorted data structures that can be quickly searched (O(log n) complexity). -> set, multiset, map, multimap
  - Unordered Associative Containers: implement unordered data structures that can be quickly searched. -> unordered_set, unordered_multiset, unordered_map, unordered_multimap

The C++ Standard Template Library: STL (Cont.)

- Advantages of the C++ Standard Template Library: STL
  - Efficient algorithms: Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.
  - Improved code readability: The STL provides a consistent and well-documented way of working with data, which can make your code easier to understand and maintain.
  - Large community of users: The STL is widely used, which means that there is a large community of developers who can provide support and resources, such as tutorials and forums.

The C++ Standard Template Library: STL (Cont.)

- Disadvantages of the C++ Standard Template Library: STL
  - Learning curve: The STL can be difficult to learn, especially for beginners, due to its complex syntax and use of advanced features like iterators.
  - Lack of control: When using the STL, you have to rely on the implementation provided by the library, which can limit your control over certain aspects of your code.
  - Performance: In some cases, using the STL can result in slower execution times compared to custom code, especially when dealing with small amounts of data.

Array class in C++ (Cont.)

- The array is a collection of homogeneous objects and this array container is defined for constant size arrays or (static size).
- In order to utilize arrays, we need to include the array header: #include <array>
- array <type, size> var_name;

Array class in C++ (Cont.)

- Operations on array :-
  - at() :- This function is used to access the elements of array. O(1)
  - get() :- This function is also used to access the elements of array. O(1)
  - operator[] :- This is similar to C-style arrays. This method is also used to access array elements. O(1)
  - front() :- This returns reference to the first element of array. O(1)
  - back() :- This returns reference to the last element of array. O(1)

Array class in C++ (Cont.)

- Operations on array :-
  - size() :- It returns the number of elements in array. O(1)
  - max_size() :- It returns the maximum number of elements array can hold. O(1)
  - swap() :- The swap() swaps all elements of one array with other. O(n)
  - empty() :- This function returns true when the array size is zero else returns false. O(1)
  - fill() :- This function is used to fill the entire array with a particular value. O(n)

Vector

- Vectors are same as dynamic arrays with the ability to resize itself automatically when an element is inserted or deleted, with their storage being handled automatically by the container.
- Vector elements are placed in contiguous storage so that they can be accessed and traversed using iterators.
- In vectors, data is inserted at the end.

Vector

- Removing the last element takes only constant time because no resizing happens.
- Inserting and erasing at the beginning or in the middle is linear in time.
- In order to utilize vector, we need to include the array header: #include <vector>
- vector <type> var_name;

Vector (Cont.)

- Operations on vector :-
  - reference operator [i] – Returns a reference to the element at position ‘i’ in the vector. O(1)
  - at(i) – Returns a reference to the element at position ‘i’ in the vector. O(1)
  - front() – Returns a reference to the first element in the vector. O(1)
  - back() – Returns a reference to the last element in the vector. O(1)

Vector (Cont.)

- Operations on vector :-
  - assign() – It assigns new value to the vector elements by replacing old ones. O(n)
  - push_back() – It push the elements into a vector from the back. O(1)
  - pop_back() – It is used to pop or remove elements from a vector from the back. O(1)
  - swap() – It is used to swap the contents of one vector with another vector of same type and size. O(1)
  - clear() – It is used to remove all the elements of the vector container. O(n)

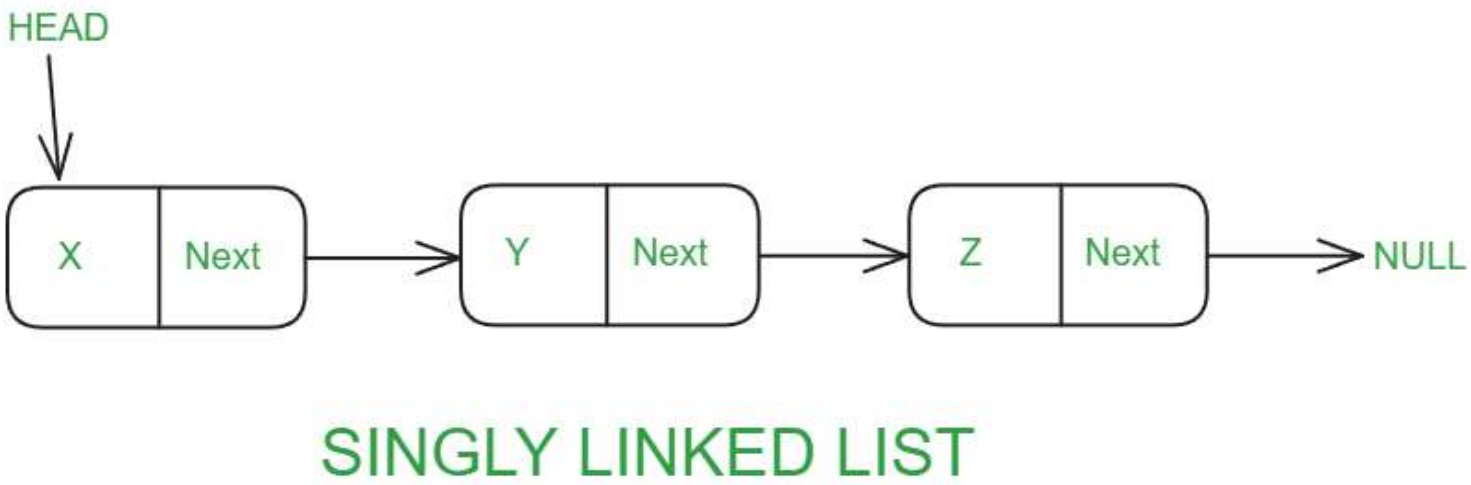
Vector (Cont.)

- Operations on vector :-
  - size() – Returns the number of elements in the vector. O(1)
  - max_size() – Returns the maximum number of elements that the vector can hold. O(1)
  - capacity() – Returns the size of the storage space currently allocated to the vector expressed as number of elements. O(1)
  - resize(n) – Resizes the container so that it contains ‘n’ elements. O(n)--Linear on the number of elements inserted/erased
  - empty() – Returns whether the container is empty. O(1)
  - shrink_to_fit() – Reduces the capacity of the container to fit its size and destroys all elements beyond the capacity. O(n)
  - reserve() – Requests that the vector capacity be at least enough to contain n elements. O(n)

List

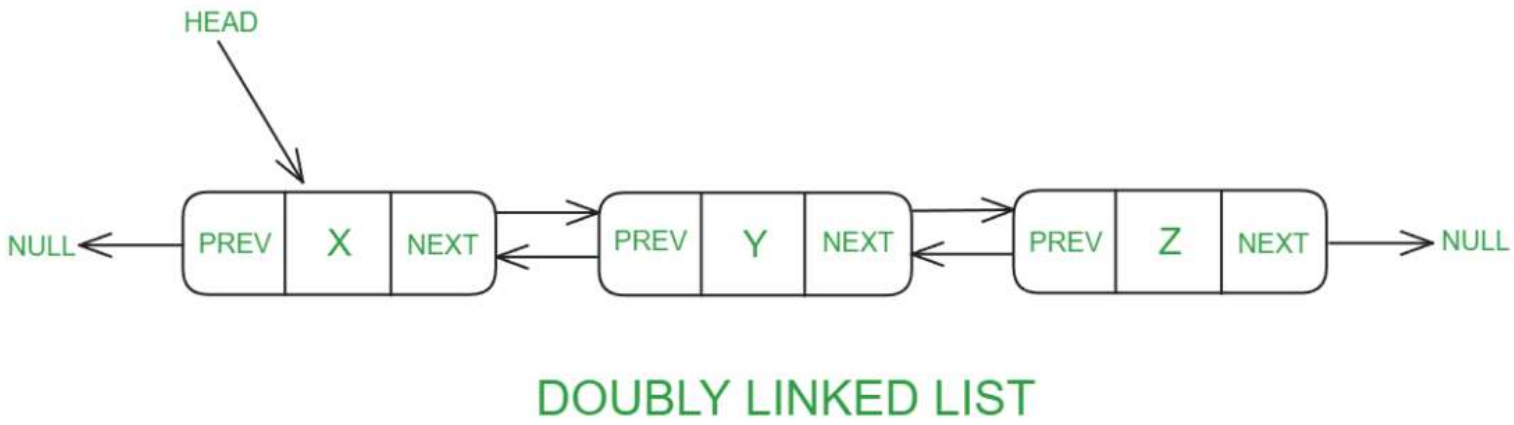
- List has slow traversal, but once a position has been found, insertion and deletion are quick.
- Normally, when we say a List, we talk about doubly linked list.
- For implementing a singly linked list, we use forward list.
- #include <list>
- list <type> var_name;

List (Cont.)



Singly linked list [11]

List (Cont.)



Doubly linked list [11]

List (Cont.)

- Operators on list :-
  - front() – Returns the value of the first element in the list. O(1)
  - back() – Returns the value of the last element in the list . O(1)
  - push_front(g) – Adds a new element ‘g’ at the beginning of the list . O(1)
  - push_back(g) – Adds a new element ‘g’ at the end of the list. O(1)
  - pop_front() – Removes the first element of the list, and reduces size of the list by 1. O(1)
  - pop_back() – Removes the last element of the list, and reduces size of the list by 1. O(1)
  - reverse() – Reverses the list. O(n)

List (Cont.)

- Operators on list :-
  - assign() – Assigns new elements to list by replacing current elements and resizes the list. O(n)
  - remove() – Removes all the elements from the list, which are equal to given element. O(n)
  - size() – Returns the number of elements in the list. O(1)
  - list resize() – Used to resize a list container. O(n)
  - empty() – Returns whether the list is empty(1) or not(0). O(1)

List (Cont.)

- Operators on list :-
  - clear() – clear() function is used to remove all the elements of the list container, thus making it size 0. O(n)
  - operator= – This operator is used to assign new contents to the container by replacing the existing contents. O(n)
  - swap() – This function is used to swap the contents of one list with another list of the same type. O(1)

Forward_List

- Forward list in STL implements singly linked list.
- Introduced from C++11, forward list are more useful than other containers in insertion, removal, and moving operations (like sort) and allow time constant insertion and removal of elements.
- It differs from the list by the fact that the forward list keeps track of the location of only the next element while the list keeps track of both the next and previous elements, thus increasing the storage space required to store each element.
- #include <forward_list>
- forward_list <type> var_name;

Forward_List (Cont.)

- The drawback of a forward list is that it cannot be iterated backward and its individual elements cannot be accessed directly.
- Forward List is preferred over the list when only forward traversal is required (same as the singly linked list is preferred over doubly linked list) as we can save space.
- Some example cases are, chaining in hashing, adjacency list representation of the graph, etc.

Forward_List (Cont.)

- Operators on forward_list :-
  - assign() – This function is used to assign values to the forward list, its other variant is used to assign repeated elements. O(n)
  - push_front() – This function is used to insert the element at the first position on forward list. The value from this function is copied to the space before first element in the container. The size of forward list increases by 1. O(1)
  - pop_front() – This function is used to delete the first element of the list. O(1)

Forward_List (Cont.)

- Operators on forward_list :-
  - remove() – This function removes the particular element from the forward list mentioned in its argument. O(n)
  - clear() This function deletes all the elements from the list. O(n)
  - front() – This function is used to reference the first element of the forward list container. O(1)
  - resize() – Changes the size of forward_list. O(n)
  - reverse() – Reverses the order of the elements present in the forward_list. O(n)

Queue

- In C++, **queue** container follows the **FIFO** (**F**irst **I**n **F**irst **O**ut) order of insertion and deletion.
- According to it, the elements that are inserted first should be removed first.
- This is possible by inserting elements at one end (called **back**) and deleting them from the other end (called **front**) of the data structure.
- #include <queue>
- queue<data_type> var_name;

Queue (Cont.)

- Operators on queue :-
  - front() - Access the front element of the queue. O(1)
  - back() - Access the end element of the queue. O(1)
  - empty() - Check whether a queue is empty or not. O(1)
  - size() - Returns the number of elements in the queue. O(1)
  - push() - Adding an element at the back of the queue. O(1)
  - pop() - Delete the front element of the queue. O(1)
  - swap() - Swap two queues. O(1)

Priority Queue

- In C++, **priority queue** is a type of queue in which there is some priority assigned to the elements.
- According to this priority, elements are removed from the queue.
- By default, the value of the element being inserted is considered as priority.
- Higher its value, higher its priority.
- #include <queue>
- priority_queue<data_type> var_name;

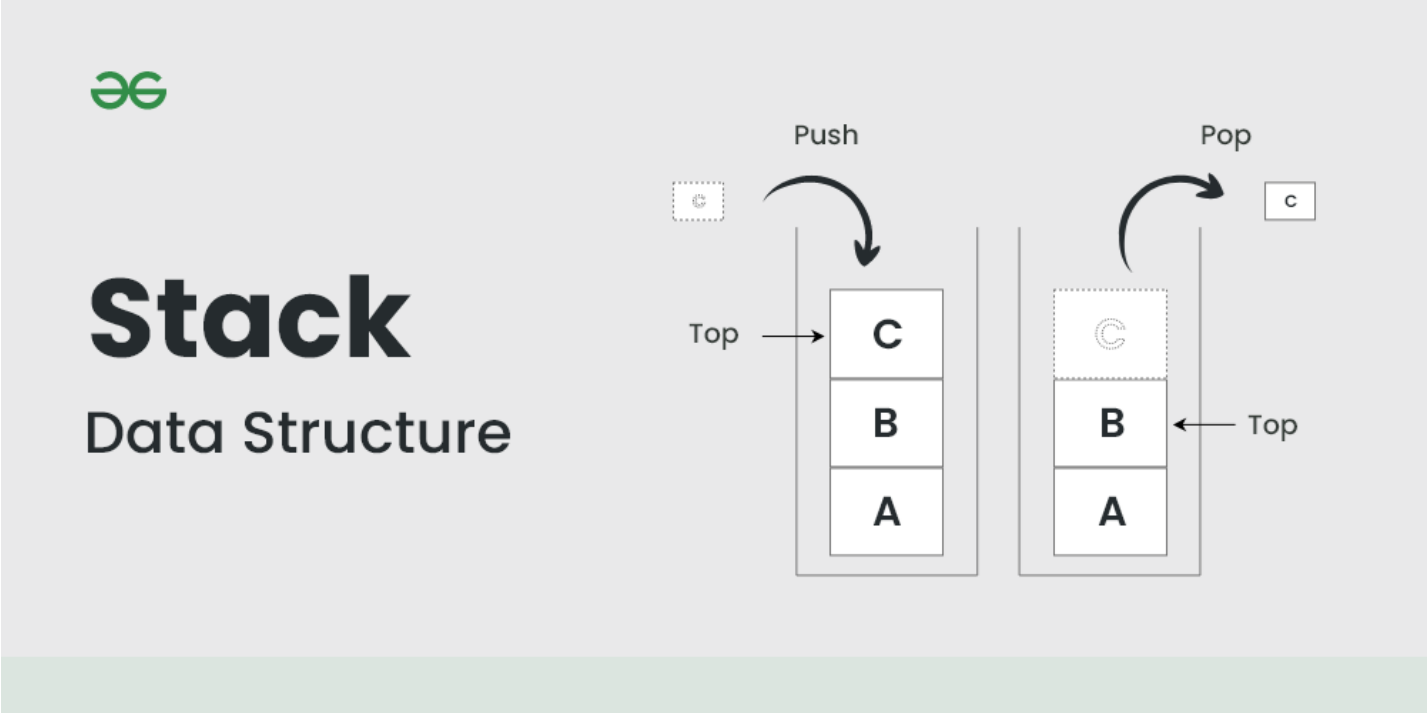
Priority Queue (Cont.)

- Operators on priority queue :-
  - empty() - Check whether the priority queue is empty or not. O(1)
  - size() - Returns the number of elements in the priority queue. O(1)
  - top() - Returns top element of the priority queue. O(1)
  - push() - Adds the element into the priority queue. O(log n)
  - pop() - Deletes the top element of the priority queue. O(log n)
  - swap() - Used to swap the contents of two priority queues provided the queues must be of the same type. O(1)

Stack

- A **Stack** is a linear data structure that follows a particular order in which the operations are performed. The order may be **LIFO(Last In First Out)** or **FILO(First In Last Out)**. **LIFO** implies that the element that is inserted last, comes out first and **FILO** implies that the element that is inserted first, comes out last.
- #include <stack>
- stack<data_type> var_name;

Stack (Cont.)



Stack [7]

Stack (Cont.)

- Operators on stack :-
  - push() - Add an element top of the stack. O(1)
  - top() - Access the top element of the stack. O(1)
  - pop() - Remove the top element of the stack. O(1)
  - stack<type> stack_name2(stack_name1) – creating a copy of the stack_name1 O(n)
  - size() - Returns the number of elements in the stack. O(1)
  - empty() - Checks if the stack is empty. O(1)
  - swap() – swap two stacks. O(1)

References

[1] Anonymous, “The C++ Standard Template Library (STL)”, Available at: https://www.geeksforgeeks.org/the-c-standard-template-library-stl/, Accessed Feb. 26, 2024.

[2] Anonymous, “Array class in C++”, Available at: https://www.geeksforgeeks.org/array-class-c/, Accessed Feb. 26, 2024.

[3] Estenger, “Vector”, Available at: https://www.geeksforgeeks.org/vector-in-cpp-stl/, Accessed Feb. 26, 2024.

[4] Anonymous, “Containers”, Available at: https://cplusplus.com/reference/stl/, Accessed Feb. 26, 2024.

[5] Anonymou, “List”, Available at: https://www.geeksforgeeks.org/list-cpp-stl/, Accessed Feb. 28, 2024.

References (Cont.)

[6] Manjeet Singh, “Forward List in C++| Set 1 (Introduction and Important Functions)”, Available https://www.geeksforgeeks.org/forward-list-c-set-1-introduction-important-functions/, Accessed Feb. 28, 2024.

[7] Geeksforgeeks, “Stack Data Structure”, Available https://www.geeksforgeeks.org/stack-data-structure/, Accessed March 23, 2025.

[8] Geeksforgeeks, “Stack in C++ STL”, Available https://www.geeksforgeeks.org/stack-in-cpp-stl/, Accessed March 23, 2025.

[9] Geeksforgeeks, “Queue in C++ STL”, Available https://www.geeksforgeeks.org/queue-cpp-stl/, Accessed March 23, 2025.

[10] Geeksforgeeks, “Priority Queue in C++ STL”, Available https://www.geeksforgeeks.org/priority-queue-in-cpp-stl/, Accessed March 23, 2025.

[11] Geeksforgeeks, “Linked List in C++”, Available https://www.geeksforgeeks.org/cpp-linked-list/, Accessed March 23, 2025.